OptiCrop: Smart Agricultural Production Optimization Engine

Project Description:

OptiCrop harnesses machine learning to provide intelligent, data-driven crop recommendations, offering farmers a fast and personalized way to decide what to plant. The platform includes a Crop Recommendation module, which analyzes soil nutrients (Nitrogen, Phosphorus, Potassium), soil pH, and weather conditions (temperature, humidity, rainfall) to predict the single most suitable crop, and a Confidence Score display, which shows how certain the model is about its recommendation.

Utilizing a trained scikit-learn classification model, OptiCrop processes farmer-submitted form data to deliver a fast, reliable crop suggestion. Built with Flask and deployed publicly via Vercel, the platform ensures a simple, low-friction experience for farmers with limited technical expertise. With client-side input validation on every field, OptiCrop empowers farmers, agricultural researchers, and policymakers to make informed, evidence-based planting decisions.

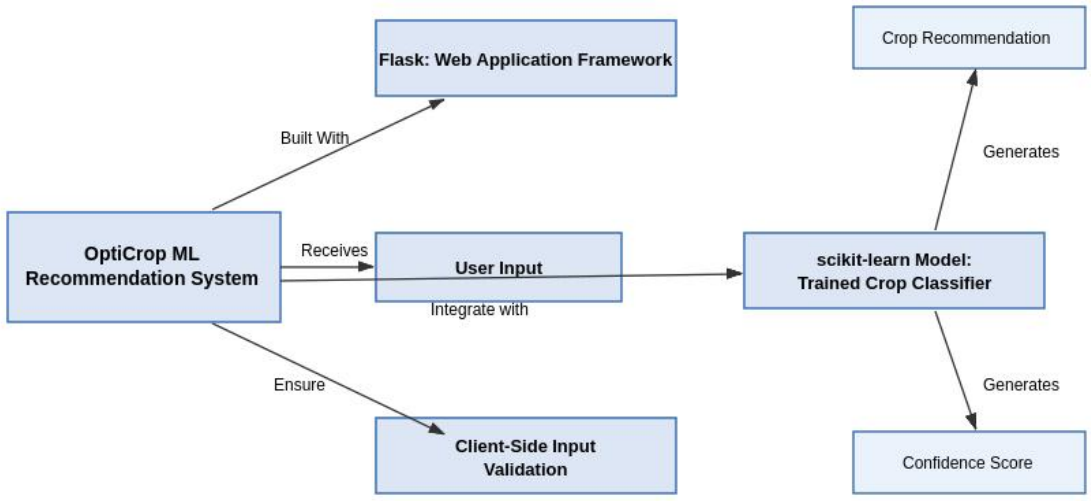
Scenarios

Scenario 1: A farmer preparing for the sowing season enters their soil's Nitrogen, Phosphorus, Potassium, and pH readings along with local temperature, humidity, and rainfall. The system analyzes the data and returns the single best-suited crop along with a confidence score, all within seconds.

Scenario 2: A farmer who already has a crop in mind wants to verify it's a good fit for their land. By entering the same soil and climate values, they can compare the system's actual recommendation against their intended crop to assess suitability before investing in seeds and inputs.

Scenario 3: An agricultural researcher or policymaker reviews recommendation patterns across many farmers' submissions to identify which soil and climate conditions correlate with which crops, supporting evidence-based, data-driven agricultural planning at a regional scale.

Technical Architecture:



Description: OptiCrop uses a simple, modular architecture to deliver fast, ML-driven crop recommendations. The frontend provides a form-based interface built with HTML and CSS (rendered via Flask/Jinja2 templates), while the Flask backend handles input parsing and passes the data directly to a pre-trained scikit-learn model. The model returns a predicted crop and confidence score, which the backend renders back to the farmer. Deployment is managed as a serverless function on Vercel.

(Note: OptiCrop does not use a live database – the trained model and label encoder are loaded from serialized .pkl files, so no ER Diagram is applicable.)

Pre-requisites:

- **Flask Framework Knowledge**: Flask Documentation (https://flask.palletsprojects.com)
- **scikit-learn / Machine Learning Familiarity**: scikit-learn Documentation (https://scikit-learn.org)
- **HTML & CSS Skills**: W3Schools HTML/CSS Tutorials
- **Python Programming Proficiency**: Python Documentation (https://docs.python.org)
- **Version Control with Git**: Git Documentation
- **Development Environment Setup**: Flask Installation Guide
- **Vercel CLI for Public Deployment**: Vercel Documentation (https://vercel.com/docs)

Project Workflow:

Milestone 1: Model Selection and Architecture

In this milestone, the focus is on selecting an appropriate classification algorithm for crop prediction and defining how the frontend, backend, and model will interact.

Activity 1.1: Prepare the Dataset

- Gather a labeled dataset containing Nitrogen, Phosphorus, Potassium, temperature, humidity, pH, rainfall, and the corresponding recommended crop label.
- Clean the data and split it into training and test sets.

Activity 1.2: Research and Select the Classification Model

- Compare candidate scikit-learn classifiers (e.g., Random Forest, Decision Tree, Naive Bayes) on accuracy and inference speed for this dataset size.
- Select the model that best balances prediction accuracy with fast, lightweight inference suitable for a serverless deployment.
- Serialize the trained model and label encoder using pickle for use in production (crop_model.pkl, label_encoder.pkl).

Activity 1.3: Define the Architecture of the Application

- Draft an architectural diagram covering the frontend (HTML/CSS via Jinja2), the Flask backend, and the model integration (see Technical Architecture above).
- Detail frontend functionality: the farmer enters soil and climate values into a single form.
- Outline backend responsibilities: the Flask app parses form data, builds a feature array, and calls the model's predict() and predict_proba() methods.

Activity 1.4: Set Up the Development Environment

```
pip install Flask numpy scikit-learn pandas gunicorn
```

- Create the project directory structure: api/ (Flask app), templates/ (HTML), static/ (CSS), and model/ (serialized model files).

Milestone 2: Core Functionalities Development

Activity 2.1: Develop Core Features

- Farmer Input Form: collects Nitrogen, Phosphorus, Potassium, pH, temperature, humidity, and rainfall, each with client-side min/max validation.
- Crop Recommendation Engine: loads the trained model and label encoder once at startup and uses them to predict a crop from submitted input.
- Confidence Score Display: reports the model's prediction probability (rounded to 1 decimal place) alongside the recommended crop.

Activity 2.2: Implement the Flask Backend

- Define routes in api/index.py for the home page ("/") and the prediction endpoint ("/predict").
- Process form input using request.form and convert each field to a float.
- Build a NumPy feature array in the exact order the model expects: N, P, K, temperature, humidity, ph, rainfall.

Milestone 3: api/index.py Development

Milestone 3 focuses on the core logic of api/index.py, which serves as the backbone of the OptiCrop application: loading the model, handling requests, and returning predictions.

Activity 3.1: Writing the Main Application Logic

- Load the trained model and label encoder once when the app starts, so every request reuses the same in-memory objects:

```
with open(os.path.join(ROOT_DIR, "model/crop_model.pkl"), "rb") as f:
    model = pickle.load(f)

with open(os.path.join(ROOT_DIR, "model/label_encoder.pkl"), "rb") as f:
    le = pickle.load(f)
```

- Home route – renders the input form:

```
@app.route("/")
def home():
    return render_template("index.html")
```

- Prediction route – parses form input, runs inference, and renders the result:

```
@app.route("/predict", methods=["POST"])
def predict():
    N=float(request.form["nitrogen"])
    P=float(request.form["phosphorus"])
    K=float(request.form["potassium"])
    temperature=float(request.form["temperature"])
    humidity=float(request.form["humidity"])
    ph=float(request.form["ph"])
    rainfall=float(request.form["rainfall"])

    data=np.array([[N,P,K,temperature,humidity,ph,rainfall]])
```

```
    pred=model.predict(data)
    prob=model.predict_proba(data)

    crop=le.inverse_transform(pred)[0]
    confidence=round(prob.max()*100,1)

    return render_template(
        "result.html",
        crop=crop,
        confidence=confidence
    )
```

- Note: unlike an AJAX/JSON-based API, this route performs a full server-side render of result.html and returns HTML directly – there is no separate JSON response contract to document here.

Milestone 4: Frontend Development

Milestone 4 focuses on the farmer-facing input form and results page. Unlike single-page JavaScript-driven apps, OptiCrop uses a traditional multi-page Flask/Jinja2 flow: submitting the form triggers a full page navigation to the result page.

Activity 4.1: Designing the User Interface

- Build index.html as the main input form, organized into two sections: "Soil Nutrients" (Nitrogen, Phosphorus, Potassium, pH) and "Weather Conditions" (temperature, humidity, rainfall).
- Apply min/max/step constraints on every numeric input (e.g., Nitrogen: 0-140 kg/ha, pH: 0-14) so the browser blocks clearly invalid values before submission.
- Style the form and results page using a shared static/style.css stylesheet.

Activity 4.2: Handling Form Submission

- The form uses a standard HTML <form action="/predict" method="POST"> element – no JavaScript or Fetch API is required, since Flask handles the full page render server-side.
- On submission, Flask's predict() route processes the data and renders result.html with the predicted crop and confidence score already embedded, which the browser then displays as a normal page load.

Milestone 5: Deployment

Milestone 5 covers deploying OptiCrop first locally to verify correct operation, then publicly via Vercel.

Activity 5.1: Preparing the Application for Local Deployment

- Create and activate a virtual environment, then install dependencies from requirements.txt:

```
python -m venv venv
venv\Scripts\activate      (Windows)   or   source venv/bin/activate   (Mac/Linux)
pip install -r requirements.txt
```

- Ensure model/crop_model.pkl and model/label_encoder.pkl are present in the model/ folder before starting the app.

Activity 5.2: Local Testing and Verification

```
python api/index.py
```

- Open a browser at the local address shown in the terminal and submit sample soil/climate values across a range of inputs to confirm the recommendation and confidence score render correctly.

Activity 5.3: Public Deployment via Vercel
Vercel hosts OptiCrop as a serverless Python function, making the app accessible from anywhere without managing a traditional server.

- Install the Vercel CLI:

```
npm install -g vercel
```

- vercel.json is already configured to route all incoming requests to api/index.py:

```
{
  "version": 2,
  "builds": [
    { "src": "api/index.py", "use": "@vercel/python" }
  ],
  "routes": [
    { "src": "/(.*)", "dest": "api/index.py" }
  ]
}
```

- Deploy with a single command from the project root:

```
vercel --prod
```

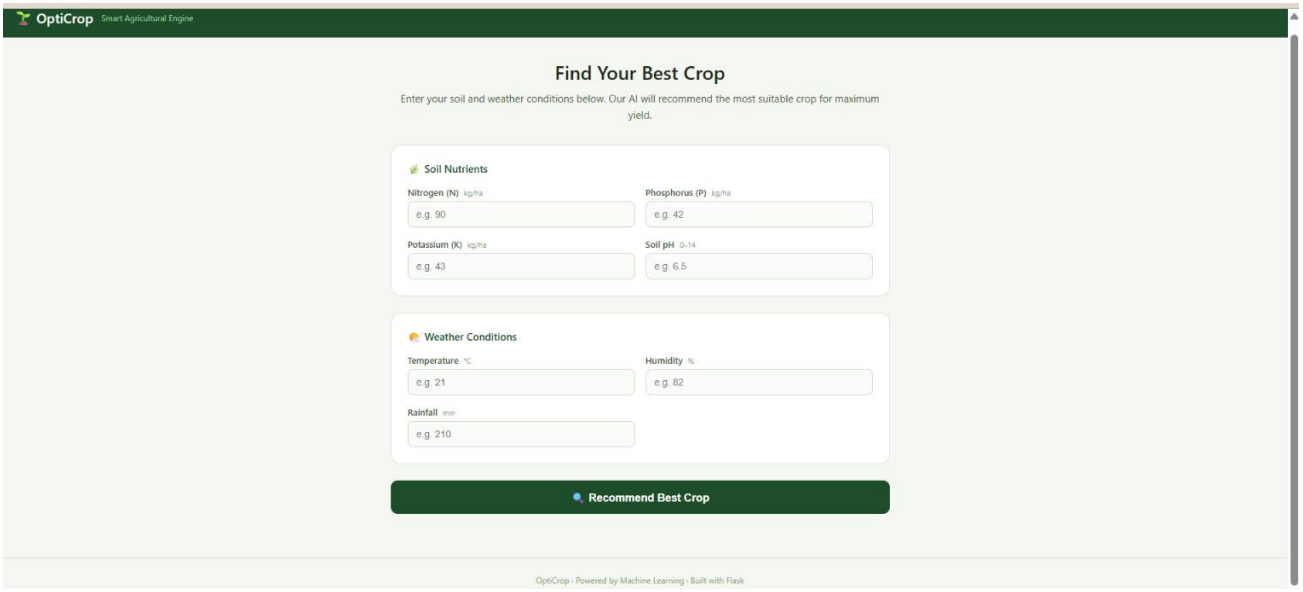
- Vercel builds the app and prints the live production URL in the terminal, e.g.: https://opticrop-indol.vercel.app/
- Share this URL with anyone who needs to access OptiCrop directly from their browser – no installation required.

Important Notes:

- Vercel serverless functions may experience "cold starts" – the first request after a period of inactivity can take several seconds longer while the function initializes and reloads the model (see the project's Performance Testing document for measured cold-start impact).
- Because there is no live database, redeploying the app (vercel --prod) is all that's needed to update the model – simply replace the .pkl files and redeploy.

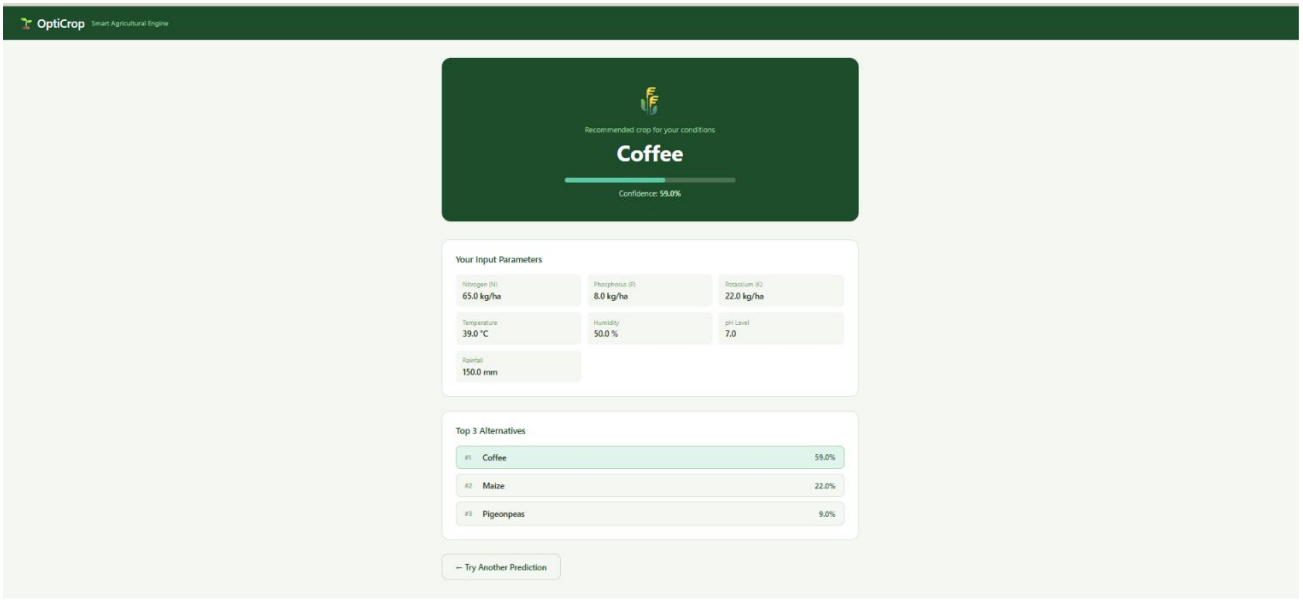
Exploring the Website's Web Pages:

Home / Input Page:



Description: The home page presents the farmer with a single form divided into “Soil Nutrients” (Nitrogen, Phosphorus, Potassium, pH) and “Weather Conditions” (temperature, humidity, rainfall) sections, each field constrained to a realistic min/max range, followed by a “Recommend Best Crop” submit button.

Result Page:



Description: After submitting the form, the farmer is taken to a result page displaying the recommended crop and the model's confidence score (%), rendered server-side by Flask's result.html template.

Conclusion

OptiCrop is a machine-learning-driven platform built with Flask that streamlines crop selection for farmers. It uses a trained scikit-learn classifier to instantly recommend the most suitable crop from soil nutrient and weather data, along with a confidence score, and is deployed publicly via Vercel. The project, which included model selection, backend/frontend development, and serverless deployment, demonstrates how a focused ML model and a simple web interface can support better-informed agricultural decisions. Looking ahead, OptiCrop has clear room to grow: adding a dedicated crop-suitability check (verifying a chosen crop against given conditions), a researcher/policymaker analytics dashboard, live weather API integration to reduce manual data entry, and basic input validation/error handling on the server side. With these additions, OptiCrop is well positioned to evolve from a single-recommendation tool into a more complete crop-planning assistant for farmers, researchers, and policymakers alike.